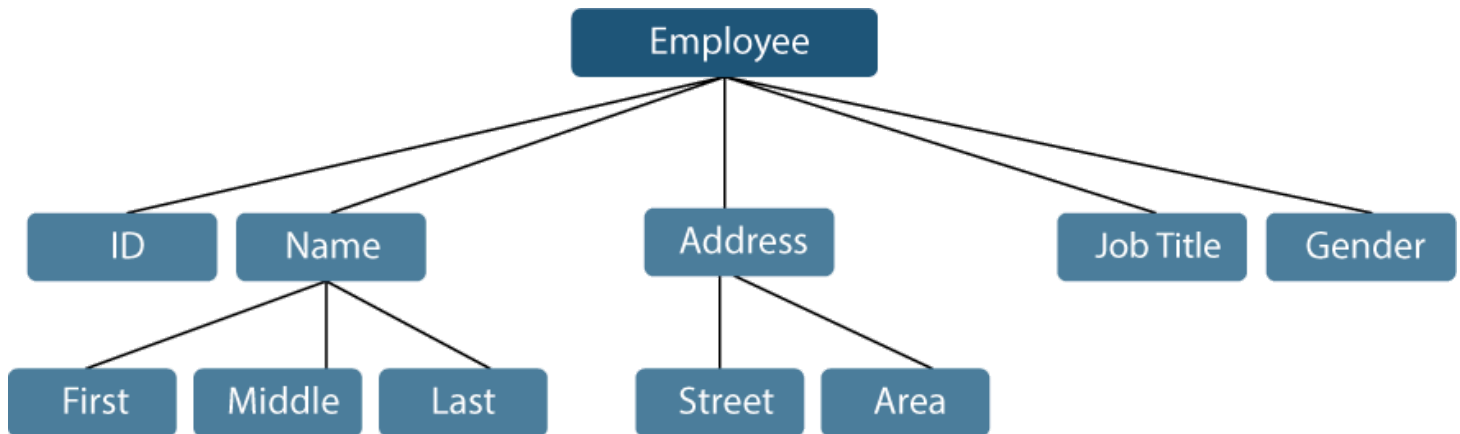


An Introduction to Data Structures

Data is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values. The data items are then classified into sub-items, which is the group of items that are not known as the simple primary form of the item.



What is Data Structure?

Data Structure is a branch of Computer Science. The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program. Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required. The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

The scope of a particular data model depends on two factors:

1. First, it must be loaded enough into the structure to reflect the definite correlation of the data with a real-world object.
2. Second, the formation should be so straightforward that one can adapt to process the data efficiently whenever necessary. Some examples of Data Structures are Arrays, Linked Lists, Stack, Queue, Trees, etc.

Understanding the Need for Data Structures

As applications are becoming more complex and the amount of data is increasing every day, which may lead to problems with data searching, processing speed, multiple requests handling, and many more. Data Structures support different methods to organize, manage, and store data efficiently. With the help of Data Structures, we can easily traverse the data items. Data Structures provide Efficiency, Reusability, and Abstraction.

Why should we learn Data Structures?

1. Data Structures and Algorithms are two of the key aspects of Computer Science.
2. Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data meaningfully.
3. Learning Data Structures and Algorithms will help us become better Programmers.
4. We will be able to write code that is more effective and reliable.
5. We will also be able to solve problems more quickly and efficiently.

Basic Terminologies related to Data Structures

Data Structures are the building blocks of any software or program. Selecting the suitable data structure for a program is an extremely challenging task for a programmer.

The following are some fundamental terminologies used whenever the data structures are involved:

1. **Data:** We can define data as an elementary value or a collection of values. For example, the Employee's name and ID are the data related to the Employee.
2. **Data Items:** A Single unit of value is known as Data Item.
3. **Group Items:** Data Items that have subordinate data items are known as Group Items. For example, an employee's name can have a first, middle, and last name.
4. **Elementary Items:** Data Items that are unable to divide into sub-items are known as Elementary Items. For example, the ID of an Employee.
5. **Entity and Attribute:** A class of certain objects is represented by an Entity. It consists of different Attributes. Each Attribute symbolizes the specific property of that Entity. For example,

Attributes	ID	Name	Gender	Job Title
Values	1234	Stacey M. Hill	Female	Software Developer

Entities with similar attributes form an **Entity Set**. Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the specific attribute.

The term "information" is sometimes utilized for data with given attributes of meaningful or processed data.

1. **Field:** A single elementary unit of information symbolizing the Attribute of an Entity is known as Field.
2. **Record:** A collection of different data items are known as a Record. For example, if we talk about the employee entity, then its name, id, address, and job title can be grouped to form the record for the employee.
3. **File:** A collection of different Records of one entity type is known as a File. For example, if there are 100 employees, there will be 25 records in the related file containing data about each employee.

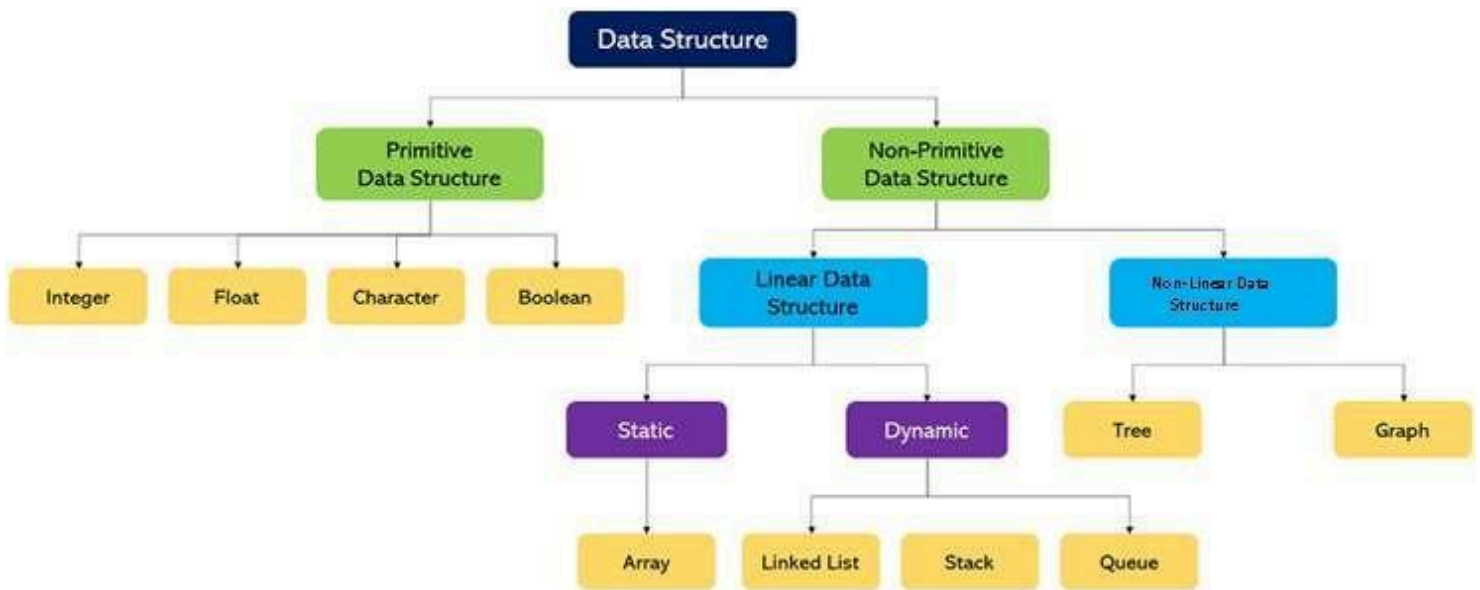
Classification of Data Structures

A Data Structure delivers a structured set of variables related to each other in various ways. It forms the basis of a programming tool that signifies the relationship between the data elements and allows programmers to process the data efficiently.

We can classify Data Structures into two categories:

1. Primitive Data Structure
2. Non-Primitive Data Structure

The following figure shows the different classifications of Data Structures.



Primitive Data Structures

1. **Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.
2. These data structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.
4. These data types are also called **Simple data types**, as they contain characters that can't be divided further

Non-Primitive Data Structures

1. **Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).
4. Based on the structure and arrangement of data, we can divide these data structures into two sub-categories
 - a. Linear Data Structures
 - b. Non-Linear Data Structures

Linear Data Structures

A data structure that preserves a linear connection among its data elements is known as a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element. However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.

Based on memory allocation, the Linear Data Structures are further classified into two types:

1. **Static Data Structures:** The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compiler time, and their size cannot be changed by the user after being compiled; however, the data stored in them can be altered. The **Array** is the best example of the Static Data Structure as they have a fixed size, and its data can be modified later.
2. **Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time, and their size varies during the

run time of the code. Moreover, the user can change the size as well as the data elements stored in these data structures at the run time of the code. **Linked Lists, Stacks, and Queues** are common examples of dynamic data structures

Types of Linear Data Structures

The following is the list of Linear Data Structures that we generally use:

1. Arrays

An **Array** is a data structure used to collect multiple data elements of the same data type into one variable. Instead of storing multiple values of the same data types in separate variable names, we could store all of them together into one variable. This statement doesn't imply that we will have to unite all the values of the same data type in any program into one array of that data type. But there will often be times when some specific variables of the same data types are all related to one another in a way appropriate for an array.

An Array is a list of elements where each element has a unique place in the list. The data elements of the array share the same variable name; however, each carries a different index number called a subscript. We can access any data element from the list with the help of its location in the list. Thus, the key feature of the arrays to understand is that the data is stored in contiguous memory locations, making it possible for the users to traverse through the data elements of the array using their respective indexes.

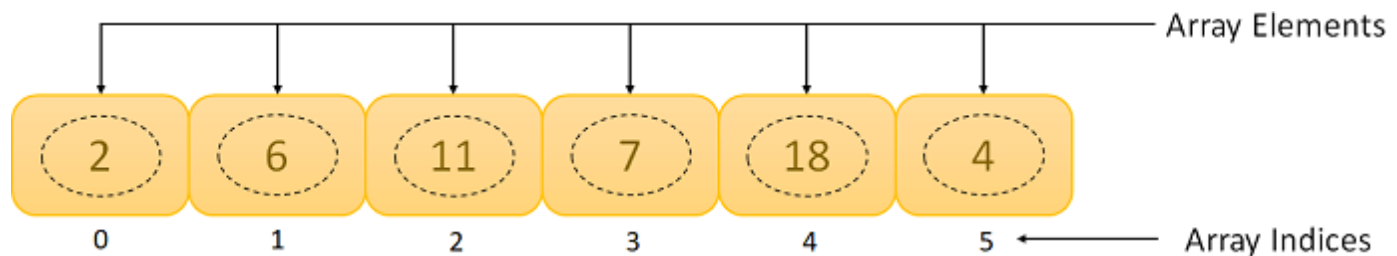


Figure 3. An Array

Arrays can be classified into different types:

- One-Dimensional Array:** An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.
- Two-Dimensional Array:** An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.
- Multidimensional Array:** We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices are per the need.

Some Applications of Array:

- We can store a list of data elements belonging to the same data type.
- Array acts as an auxiliary storage for other data structures.
- The array also helps store data elements of a binary tree of the fixed count.
- Array also acts as a storage of matrices.

2. Linked Lists

A **Linked List** is another example of a linear data structure used to store a collection of data elements dynamically. Data elements in this data structure are represented by the Nodes, connected using links or pointers. Each node contains two fields, the information field consists of the actual data, and the pointer field consists of the address of the subsequent nodes in the list. The pointer of the last node of the linked list consists of a null pointer, as it points to nothing. Unlike the Arrays, the user can dynamically adjust the size of a Linked List as per the requirements.

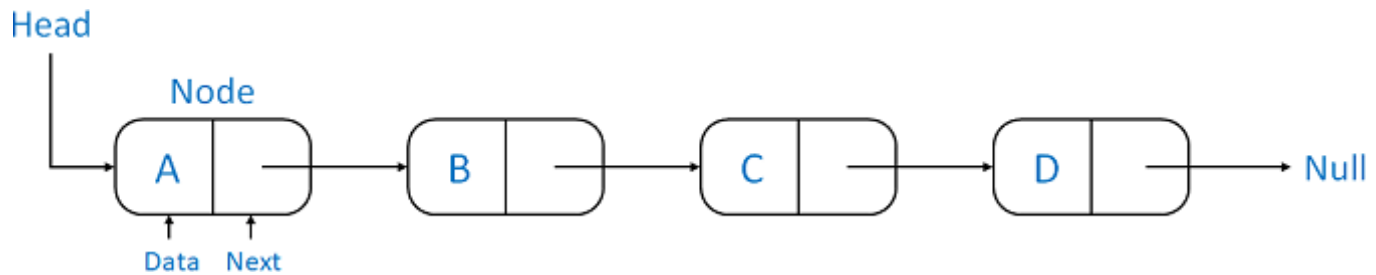


Figure 4. A Linked List

Linked Lists can be classified into different types:

- a. **Singly Linked List:** A Singly Linked List is the most common type of Linked List. Each node has data and a pointer field containing an address to the next node.
- b. **Doubly Linked List:** A Doubly Linked List consists of an information field and two pointer fields. The information field contains the data. The first pointer field contains an address of the previous node, whereas another pointer field contains a reference to the next node. Thus, we can go in both directions (backward as well as forward).
- c. **Circular Linked List:** The Circular Linked List is similar to the Singly Linked List. The only key difference is that the last node contains the address of the first node, forming a circular loop in the Circular Linked List.

Some Applications of Linked Lists:

- a. The Linked Lists help us implement stacks, queues, binary trees, and graphs of predefined size.
- b. We can also implement Operating System's function for dynamic memory management.
- c. Linked Lists also allow polynomial implementation for mathematical operations.
- d. We can use Circular Linked List to implement Operating Systems or application functions that Round Robin execution of tasks.
- e. Circular Linked List is also helpful in a Slide Show where a user requires to go back to the first slide after the last slide is presented.
- f. Doubly Linked List is utilized to implement forward and backward buttons in a browser to move forward and backward in the opened pages of a website.

3. Stacks

A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top. Stacks can be implemented with the help of contiguous memory, an Array, and non-contiguous memory, a Linked List. Real-life examples of Stacks are piles of books, a deck of cards, piles of money, and many more.

The primary operations in the Stack are as follows:

- a. **Push:** Operation to insert a new element in the Stack is termed as Push Operation.
- b. **Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.

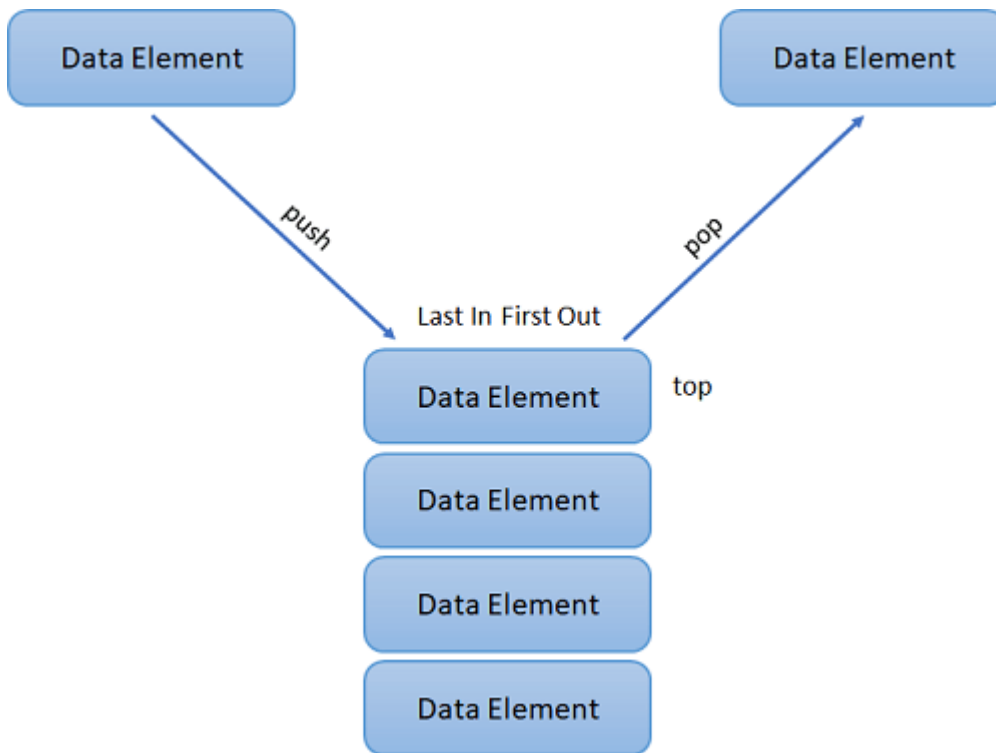


Figure 6. A Stack

Some Applications of Stacks:

- a. The Stack is used as a Temporary Storage Structure for recursive operations.
- b. Stack is also utilized as Auxiliary Storage Structure for function calls, nested operations, and deferred/postponed functions.
- c. We can manage function calls using Stacks.
- d. Stacks are also utilized to evaluate the arithmetic expressions in different programming languages.
- e. Stacks are also helpful in converting infix expressions to postfix expressions.
- f. Stacks allow us to check the expression's syntax in the programming environment.
- g. We can match parenthesis using Stacks.
- h. Stacks can be used to reverse a String.
- i. Stacks are helpful in solving problems based on backtracking.
- j. We can use Stacks in depth-first search in graph and tree traversal.
- k. Stacks are also used in Operating System functions.
- l. Stacks are also used in UNDO and REDO functions in an edit.

4. Queues

A **Queue** is a linear data structure similar to a Stack with some limitations on the insertion and deletion of the elements. The insertion of an element in a Queue is done at one end, and the removal is done at another or opposite end. Thus, we can conclude that the Queue data structure follows FIFO (First In, First Out) principle to manipulate

the data elements. Implementation of Queues can be done using Arrays, Linked Lists, or Stacks. Some real-life examples of Queues are a line at the ticket counter, an escalator, a car wash, and many more.

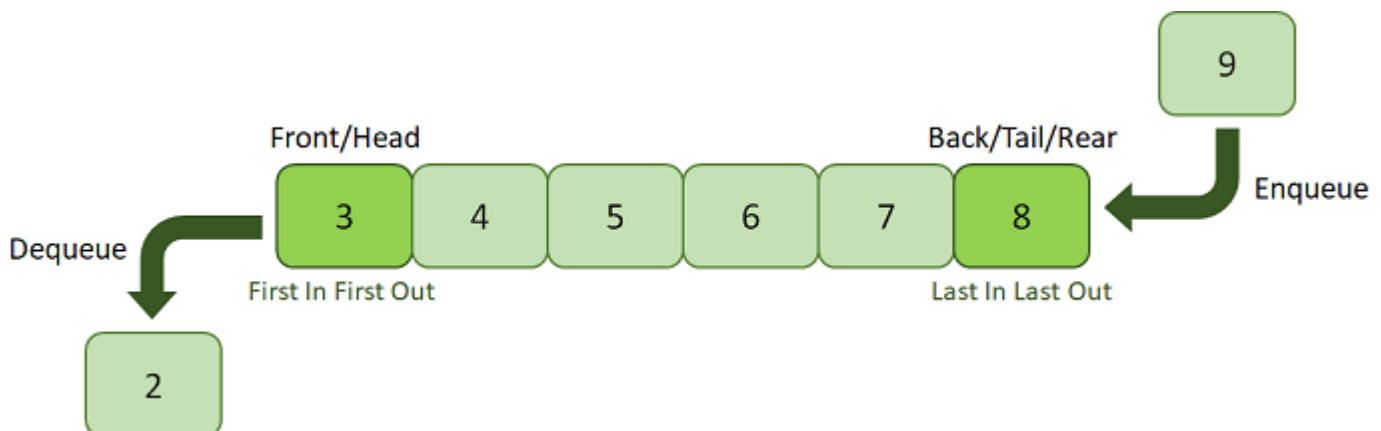


Figure 7. A Real-life Example of Queue

The above image is a real-life illustration of a movie ticket counter that can help us understand the Queue where the customer who comes first is always served first. The customer arriving last will undoubtedly be served last. Both ends of the Queue are open and can execute different operations. Another example is a food court line where the customer is inserted from the rear end while the customer is removed at the front end after providing the service they asked for.

The following are the primary operations of the Queue:

- Enqueue:** The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.
- Dequeue:** Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.



Some Applications of Queues:

- Queues are generally used in the breadth search operation in Graphs.
- Queues are also used in Job Scheduler Operations of Operating Systems, like a keyboard buffer queue to store the keys pressed by users and a print buffer queue to store the documents printed by the printer.
- Queues are responsible for CPU scheduling, Job scheduling, and Disk Scheduling.
- Priority Queues are utilized in file-downloading operations in a browser.
- Queues are also used to transfer data between peripheral devices and the CPU.
- Queues are also responsible for handling interrupts generated by the User Applications for the CPU.

Non-Linear Data Structures

Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.

Types of Non-Linear Data Structures

The following is the list of Non-Linear Data Structures that we generally use:

1. Trees

A Tree is a Non-Linear Data Structure and a hierarchy containing a collection of nodes such that each node of the tree stores a value and a list of references to other nodes (the "children").

The Tree data structure is a specialized method to arrange and collect data in the computer to be utilized more effectively. It contains a central node, structural nodes, and sub-nodes connected via edges. We can also say that the tree data structure consists of roots, branches, and leaves connected.

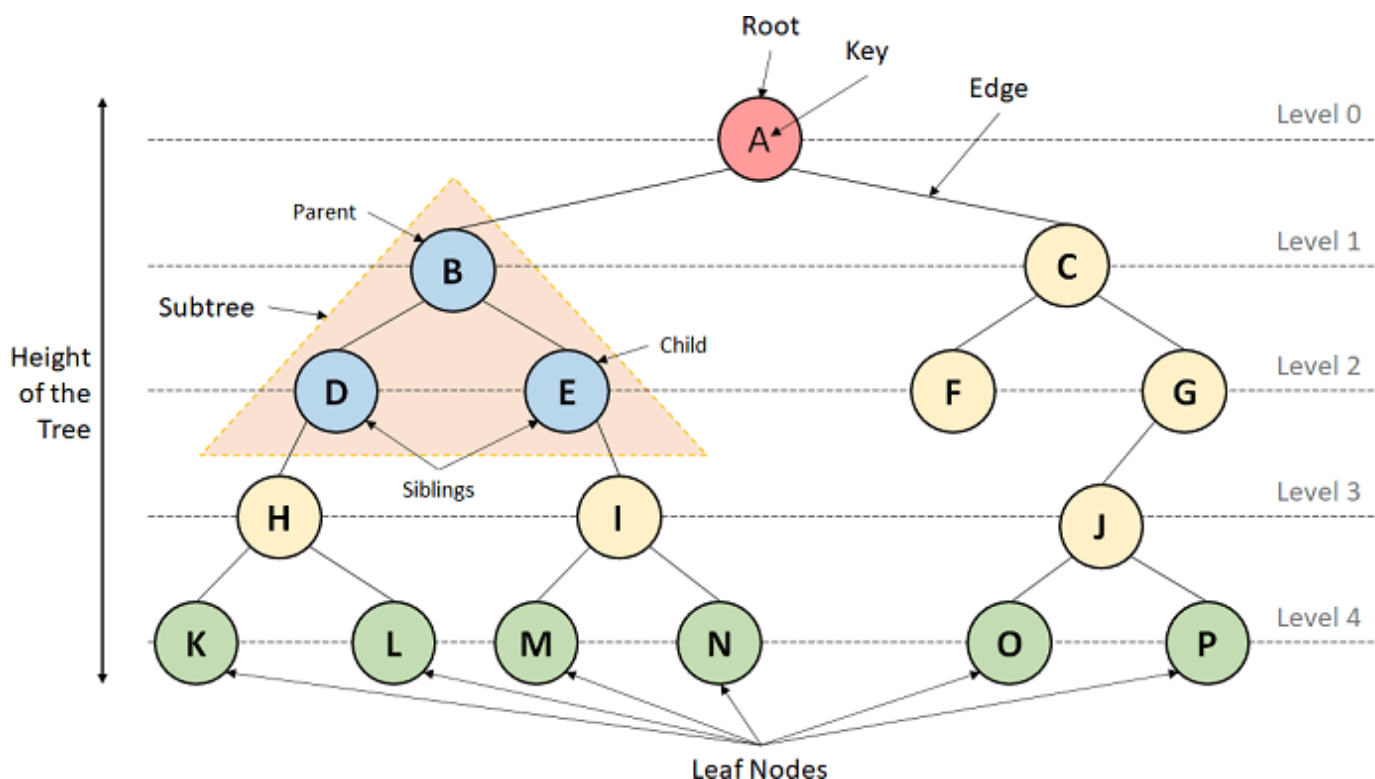


Figure 9. A Tree

Trees can be classified into different types:

- a. **Binary Tree:** A Tree data structure where each parent node can have at most two children is termed a Binary Tree.
- b. **Binary Search Tree:** A Binary Search Tree is a Tree data structure where we can easily maintain a sorted list of numbers.
- c. **AVL Tree:** An AVL Tree is a self-balancing Binary Search Tree where each node maintains extra information known as a Balance Factor whose value is either -1, 0, or +1.
- d. **B-Tree:** A B-Tree is a special type of self-balancing Binary Search Tree where each node consists of multiple keys and can have more than two children.

Some Applications of Trees:

- a. Trees implement hierarchical structures in computer systems like directories and file systems.
- b. Trees are also used to implement the navigation structure of a website.
- c. We can generate code like Huffman's code using Trees.
- d. Trees are also helpful in decision-making in Gaming applications.
- e. Trees are responsible for implementing priority queues for priority-based OS scheduling functions.
- f. Trees are also responsible for parsing expressions and statements in the compilers of different programming languages.
- g. We can use Trees to store data keys for indexing for Database Management System (DBMS).
- h. Spanning Trees allows us to route decisions in Computer and Communications Networks.
- i. Trees are also used in the path-finding algorithm implemented in Artificial Intelligence (AI), Robotics, and Video Games Applications.

2. Graphs

A Graph is another example of a Non-Linear Data Structure comprising a finite number of nodes or vertices and the edges connecting them. The Graphs are utilized to address problems of the real world in which it denotes the problem area as a network such as social networks, circuit networks, and telephone networks. For instance, the nodes or vertices of a Graph can represent a single user in a telephone network, while the edges represent the link between them via telephone.

The Graph data structure, G is considered a mathematical structure comprised of a set of vertices, V and a set of edges, E as shown below:

$$G = (V, E)$$

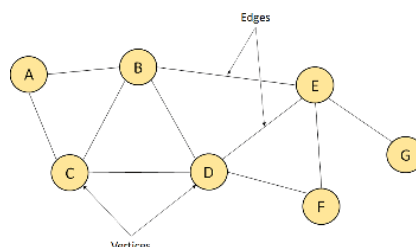


Figure 10. A Graph

The above figure represents a Graph having seven vertices A, B, C, D, E, F, G, and ten edges [A, B], [A, C], [B, C], [B, D], [B, E], [C, D], [D, E], [D, F], [E, F], and [E, G].

Depending upon the position of the vertices and edges, the Graphs can be classified into different types:

- a. **Null Graph:** A Graph with an empty set of edges is termed a Null Graph.
- b. **Trivial Graph:** A Graph having only one vertex is termed a Trivial Graph.
- c. **Simple Graph:** A Graph with neither self-loops nor multiple edges is known as a Simple Graph.
- d. **Multi Graph:** A Graph is said to be Multi if it consists of multiple edges but no self-loops.
- e. **Pseudo Graph:** A Graph with self-loops and multiple edges is termed a Pseudo Graph.
- f. **Non-Directed Graph:** A Graph consisting of non-directed edges is known as a Non-Directed Graph.
- g. **Directed Graph:** A Graph consisting of the directed edges between the vertices is known as a Directed Graph.
- h. **Connected Graph:** A Graph with at least a single path between every pair of vertices is termed a Connected Graph.
- i. **Disconnected Graph:** A Graph where there does not exist any path between at least one pair of vertices is termed a Disconnected Graph.
- j. **Regular Graph:** A Graph where all vertices have the same degree is termed a Regular Graph.
- k. **Complete Graph:** A Graph in which all vertices have an edge between every pair of vertices is known as a Complete Graph.
- l. **Cycle Graph:** A Graph is said to be a Cycle if it has at least three vertices and edges that form a cycle.
- m. **Cyclic Graph:** A Graph is said to be Cyclic if and only if at least one cycle exists.
- n. **Acyclic Graph:** A Graph having zero cycles is termed an Acyclic Graph.
- o. **Finite Graph:** A Graph with a finite number of vertices and edges is known as a Finite Graph.
- p. **Infinite Graph:** A Graph with an infinite number of vertices and edges is known as an Infinite Graph.
- q. **Bipartite Graph:** A Graph where the vertices can be divided into independent sets A and B, and all the vertices of set A should only be connected to the vertices present in set B with some edges is termed a Bipartite Graph.
- r. **Planar Graph:** A Graph is said to be a Planar if we can draw it in a single plane with two edges intersecting each other.
- s. **Euler Graph:** A Graph is said to be Euler if and only if all the vertices are even degrees.
- t. **Hamiltonian Graph:** A Connected Graph consisting of a Hamiltonian circuit is known as a Hamiltonian Graph.

Some Applications of Graphs:

- a. Graphs help us represent routes and networks in transportation, travel, and communication applications.
- b. Graphs are used to display routes in GPS.
- c. Graphs also help us represent the interconnections in social networks and other network-based applications.
- d. Graphs are utilized in mapping applications.
- e. Graphs are responsible for the representation of user preference in e-commerce applications.
- f. Graphs are also used in Utility networks in order to identify the problems posed to local or municipal corporations.
- g. Graphs also help to manage the utilization and availability of resources in an organization.
- h. Graphs are also used to make document link maps of the websites in order to display the connectivity between the pages through hyperlinks.
- i. Graphs are also used in robotic motions and neural networks.

Basic Operations of Data Structures

In the following section, we will discuss the different types of operations that we can perform to manipulate data in every data structure:

1. **Traversal:** Traversing a data structure means accessing each data element exactly once so it can be administered. For example, traversing is required while printing the names of all the employees in a department.
2. **Search:** Search is another data structure operation which means to find the location of one or more data elements that meet certain constraints. Such a data element may or may not be present in the given set of data elements. For example, we can use the search operation to find the names of all the employees who have the experience of more than 5 years.
3. **Insertion:** Insertion means inserting or adding new data elements to the collection. For example, we can use the insertion operation to add the details of a new employee the company has recently hired.
4. **Deletion:** Deletion means to remove or delete a specific data element from the given list of data elements. For example, we can use the deleting operation to delete the name of an employee who has left the job.
5. **Sorting:** Sorting means to arrange the data elements in either Ascending or Descending order depending on the type of application. For example, we can use the sorting operation to arrange the names of employees in a department in alphabetical order or estimate the top three performers of the month by arranging the performance of the employees in descending order and extracting the details of the top three.
6. **Merge:** Merge means to combine data elements of two sorted lists in order to form a single list of sorted data elements.
7. **Create:** Create is an operation used to reserve memory for the data elements of the program. We can perform this operation using a declaration statement. The creation of data structure can take place either during the following:
 - a. Compile-time
 - b. Run-timeFor example, the **malloc()** function is used in C Language to create data structure.
8. **Selection:** Selection means selecting a particular data from the available data. We can select any particular data by specifying conditions inside the loop.
9. **Update:** The Update operation allows us to update or modify the data in the data structure. We can also update any particular data by specifying some conditions inside the loop, like the Selection operation.
10. **Splitting:** The Splitting operation allows us to divide data into various subparts decreasing the overall process completion time.

Built in Data Types in C

void	As the name suggests, it holds no value and is generally used for specifying the type of function or what it returns. If the function has a void type, it means that the function will not return any value.
int	Used to denote an integer type.
char	Used to denote a character type.
float, double	Used to denote a floating point type.
int *, float *, char *	Used to denote a pointer type.

What is an Algorithm?

An algorithm is a set of guidelines or rules used to solve a problem or carry out a task. It is a methodical approach that may be used to solve a mathematical riddle, run a computer program, or carry out a task like sorting a list.

An algorithm can also be used to specify a set of guidelines for carrying out a certain activity. Algorithms can be designed to solve a wide range of problems and tasks, from the mundane to the complex.

Characteristics of Algorithm

- **Unambiguous:** Algorithms must have a clear and well-defined set of instructions that are easy to understand and follow.
- **Effective:** The algorithm must produce the correct result in a finite amount of time.
- **Feasible:** The algorithm must be physically applicable and must be able to run on a computer.
- **Input and Output:** Algorithms must take inputs and produce outputs.
- **Finite:** The algorithm must terminate after a finite number of steps and must not go into an infinite loop.
- **Deterministic:** The algorithm must produce the same output for the same input every time it is run.

Time and Space Complexity

Time complexity is defined in terms of how many times it takes to run a given algorithm, based on the length of the input. Time complexity is a measurement of how much time it takes to execute a particular algorithm because such factors as programming language, operating system, and processing power are also considered.

When an algorithm is run on a computer, it necessitates a certain amount of memory space. The amount of memory used by a program to execute it is represented by its space complexity. Because a program requires memory to store input data and temporal values while running, the space complexity is auxiliary and input space.

Asymptotic notations

The main idea of asymptotic analysis is to have a measure of the efficiency of algorithms that don't depend on machine-specific constants and don't require algorithms to be implemented and time taken by programs to be compared. Asymptotic notations are mathematical tools to represent the time complexity of algorithms for asymptotic analysis.

- O – Big Oh Notation
- Ω – Big Omega Notation
- Θ – Theta Notation
- o – Little Oh Notation
- ω – Little Omega Notation

O – Big Oh Notation: The notation $O(n)$ is the formal way to express the upper bound of an algorithm's running time. It measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

The general step wise procedure for Big-O runtime analysis is as follows:

1. Figure out what the input is and what n represents.
2. Express the maximum number of operations, the algorithm performs in terms of n .
3. Eliminate all excluding the highest order terms.
4. Remove all the constant factors.

For example, for a function **$f(n)$**

$O(f(n)) = \{ g(n) : \text{there exists } c > 0 \text{ and } n_0 \text{ such that } g(n) \leq c.f(n) \}$

Example: Let us consider a given function, **$f(n) = 4n^3 + 10n^2 + 5n + 1$** .

Considering $g(n) = n^3$

$f(n) \geq 5.g(n)$ for all the values of $n > 2$.

Hence, the complexity of $f(n)$ can be represented as $O(g(n))$, i.e. $O(n^3)$.

Big Omega Notation, Ω : The notation $\Omega(n)$ is the formal way to express the lower bound of an algorithm's running time. It measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

For example, for a function **$f(n)$**

$\Omega(f(n)) \geq \{ g(n) : \text{there exists } c > 0 \text{ and } n_0 \text{ such that } g(n) \leq c.f(n) \text{ for all } n > n_0. \}$

Example

Let us consider a given function, $f(n) = 4.n^3 + 10.n^2 + 5.n + 1$

Considering $g(n) = n^3$, $f(n) \geq 4.g(n)$ for all the values of $n > 0$.

Hence, the complexity of $f(n)$ can be represented as $\Omega(g(n))$, i.e. $\Omega(n^3)$.

Theta Notation, θ : The notation $\theta(n)$ is the formal way to express both the lower bound and the upper bound of an algorithm's running time. Some may confuse the theta notation as the average case time complexity; while big theta notation could be almost accurately used to describe the average case, other notations could be used as well.
 $\theta(f(n)) = \{ g(n) \text{ if and only if } g(n) = O(f(n)) \text{ and } g(n) = \Omega(f(n)) \text{ for all } n > n_0. \}$

Example

Let us consider a given function, $f(n) = 4.n^3 + 10.n^2 + 5.n + 1$

Considering $g(n) = n^3$, $4.g(n) \leq f(n) \leq 5.g(n)$ for all the values of n .

Hence, the complexity of $f(n)$ can be represented as $\Theta(g(n))$, i.e. $\Theta(n^3)$.

Difference Between Big Oh, Big Omega and Big Theta :			
S.No.	Big Oh (O)	Big Omega (Ω)	Big Theta (Θ)
1	It is like ($\leq$) rate of growth of an algorithm is less than or equal to a specific value.	It is like ($\geq$) rate of growth is greater than or equal to a specified value.	It is like ($=$) meaning the rate of growth is equal to a specified value.
2	The upper bound of algorithm is represented by Big O notation. Only the above function is bounded by Big O. Asymptotic upper bound is given by Big O notation.	The algorithm's lower bound is represented by Omega notation. The asymptotic lower bound is given by Omega notation.	The bounding of function from above and below is represented by theta notation. The exact asymptotic behavior is done by this theta notation.

3	Big oh (O) – Upper Bound	Big Omega (Ω) – Lower	Big Theta (Θ) – Tight
---	--------------------------	--------------------------------	--------------------------------

		Bound	Bound
4	It is define as upper bound and upper bound on an algorithm is the most amount of time required (the worst case performance).	It is define as lower bound and lower bound on an algorithm is the least amount of time required (the most efficient way possible, in other words best case).	It is define as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.
5	Mathematically: Big Oh is $0 \leq f(n) \leq Cg(n)$ for all $n \geq n_0$	Mathematically: Big Omega is $0 \leq Cg(n) \leq f(n)$ for all $n \geq n_0$	Mathematically – Big Theta is $0 \leq C_2g(n) \leq f(n) \leq C_1g(n)$ for $n \geq n_0$

Time Space Trade Off:

It is a way of solving a problem or calculation in less time by using more storage space (or memory), or by solving a problem in very little space by spending a long time. Example involving the concept of Time Space Tradeoff:

- 1.If data is stored uncompressed, it takes more space but less time.
2. Storing only the source and rendering it as an image every time the page is requested would be trading time for space. More time used but less space.

Abstract Data Type (ADT):

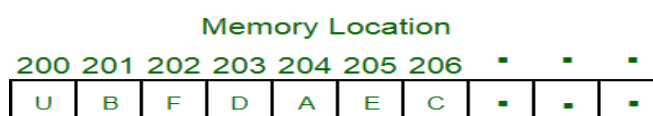
Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations. The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. It is called “abstract” because it gives an implementation-independent view.

Arrays:

An array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. This makes it easier to calculate the position of each element by simply adding an offset to a base value, i.e., the memory location of the first element of the array (generally denoted by the name of the array).

One Dimensional Array:

- It is a list of the variable of similar data types.
- It allows random access and all the elements can be accessed with the help of their index.
- The size of the array is fixed.
- For a dynamically sized array, vector can be used in C++.
- Representation of 1D array:



Two Dimensional Array:

- It is a list of lists of the variable of the same data type.
- It also allows random access and all the elements can be accessed with the help of their index.
- It can also be seen as a collection of 1D arrays. It is also known as the Matrix.
- Its dimension can be increased from 2 to 3 and 4 so on.
- They all are referred to as a multi-dimension array.
- The most common multidimensional array is a 2D array.
- Representation of 2 D array:

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Basis	One Dimension Array	Two Dimension Array
Definition	Store a single list of the element of a similar data type.	Store a 'list of lists' of the element of a similar data type.
Representation	Represent multiple data items as a list.	Represent multiple data items as a table consisting of rows and columns.
Declaration	The declaration varies for different programming language: 1. For C++, <i>datatype variable_name[row]</i> 2. For Java, <i>datatype [] variable_name= new datatype[row]</i>	The declaration varies for different programming language: 1. For C++, <i>datatype variable_name[row][column]</i> 2. For Java, <i>datatype [][] variable_name= new datatype[row][column]</i>
Dimension	One	Two
Size(bytes)	size of(datatype of the variable of the array) * size of the array	size of (datatype of the variable of the array)* the number of rows* the number of columns.
Address calculation.	Address of a[index] is equal to (base Address+ Size of each element of array * index).	Address of a[i][j] can be calculated in two ways row-major and column-major 1. Column Major: Base Address + Size of each element (number of rows(j-lower bound of the column)+(i-lower bound of the rows))

		2. Row Major: Base Address + Size of each element (number of columns(i-lower bound of the row)+(j-lower bound of the column))
Example	<pre>int arr[5]; //an array with one row and five columns will be created. {a , b , c , d , e}</pre>	<pre>int arr[2][5]; //an array with two rows and five columns will be created. a b c d e f g h i j</pre>

Q.-What are the merits and demerits of array data structures?

Ans: Array is a linear data structure that is a collection of similar data types. Arrays are stored in contiguous memory locations. It is a static data structure with a fixed size. It combines data of similar types. ARRAY

Applications of Array Data Structure:

Below are some applications of arrays.

- Arrays are used to implement data structures like a stack, queue, etc.
- Arrays are used for matrices and other mathematical implementations.
- Arrays are used in lookup tables in computers.
- Arrays can be used for CPU scheduling.

Advantages of Array:

1. Multiple data items of same type can be declared using same name but different indexes
2. Accessing an element becomes easy just by using index no.
3. Arrays allocate memory in contiguous memory locations, hence no extra memory is allocated.

Disadvantages:

1. Number of elements should be known in advance either declared at run time by the user or initialized at compile time
2. Insertion and deletion of elements is quite difficult
3. Size of initialized array cannot be declared only one time. It cannot be changed again and again.
4. Datatypes of elements should be same

If elements are less than size then extra memory is wasted

Generalized Formula for address Calculation

1) 1D Array

Calculating the address of any element in the 1-D array:

A 1-dimensional array (or single-dimension array) is a type of linear array. Accessing its elements involves a single subscript that can either represent a row or column index.

To find the address of an element in an array the following formula is used-

$$\text{Address of } A[I] = B + W * (I - LB)$$

Where,

I = Subset of element whose address to be found,

B = Base address,

W = Storage size of one element store in any array (in byte),

LB = Lower Limit/Lower Bound of subscript (If not specified assume zero).

Example1: Given the base address of an array A[1300 1900] as 1020 and the size of each element is 2 bytes in the memory, find the address of A[1700].

Solution:

Given:

Base address B = 1020

Lower Limit/Lower Bound of subscript LB = 1300
 Storage size of one element store in any array W = 2 Byte
 Subset of element whose address to be found I = 1700

Formula used:

Address of A[I] = B + W * (I - LB)

Solution: Address of A[1700] = 1020 + 2 * (1700 - 1300) = 1020 + 2 * (400)
 = 1020 + 800 Address of A [1700] = 1820

2. Calculate the address of any element in the 2-D array:

The 2-dimensional array can be defined as an array of arrays. The 2-Dimensional arrays are organized as matrices which can be represented as the collection of rows and columns as array[N1][N2] where N1 is the number of rows and N2 is the number of columns.

Given:

Base Address: 6500

2D Array : A[50][60]
 N1 N2

Element to be find: A[34][50]
 n1 n2

Row Major Order:

Base Address + (n1N2 + n2) * Size

Column Major Order:

Base Address + (n2N1 + n1) * Size

Example:

Rows

2D Array	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

2-D array

To find the address of any element in a **2-Dimensional** array there are the following two

ways-

1. Row Major Order
2. Column Major Order

1. Row Major Order:

Row major ordering assigns successive elements, moving across the rows and then down the next row, to successive memory locations. In simple language, the elements of an array are stored in a Row-Wise fashion.

To find the address of the element using row-major order uses the following formula:

$$\text{Address of } A[n1][n2] = B + W * ((n1 - LR) * N2 + (n2 - LC))$$

$n1$ = Row Subset of an element whose address to be found,

$n2$ = Column Subset of an element whose address to be found, B = Base address,

W = Storage size of one element store in an array (in byte),

LR = Lower Limit of row/start row index of the matrix (If not given assume it as zero),

LC = Lower Limit of column/start column index of the matrix (If not given assume it as zero),

$N2$ = Number of column given in the matrix.

Example: Given an array, `arr[1.....10][1.....15]` with base value **100** and the size of each element is **1 Byte** in memory. Find the address of `arr[8][6]` with the help of row- major order.

Solution:

Given:

Base address $B = 100$

Storage size of one element store in any array $W = 1$ Bytes

Row Subset of an element whose address to be found $n1 = 8$

Column Subset of an element whose address to be found $n2 = 6$

Lower Limit of row/start row index of matrix $LR = 1$

Lower Limit of column/start column index of matrix $= 1$

$$\begin{aligned}\text{Number of column given in the matrix } N2 &= \text{Upper Bound} - \text{Lower Bound} + 1 \\ &= 15 - 1 + 1 \\ &= 15\end{aligned}$$

Formula:

$$\text{Address of } A[n1][n2] = B + W * ((n1 - LR) * N2 + (n2 - LC))$$

Solution:

$$\begin{aligned}\text{Address of } A[8][6] &= 100 + 1 * ((8 - 1) * 15 + (6 - 1)) \\ &= 100 + 1 * ((7) * 15 + (5)) \\ &= 100 + 1 * (110)\end{aligned}$$

$$\text{Address of } A[I][J] = 210$$

2. Column Major Order:

If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order. To find the address of the element using column-major order use the following formula:

$$\text{Address of } A[n1][n2] = B + W * ((n2 - LC) * N1 + (n1 - LR))$$

$n1$ = Row Subset of an element whose address to be found,

$n2$ = Column Subset of an element whose address to be found,

found, B = Base address,

W = Storage size of one element store in any array(in byte),

LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),

$N1$ = Number of rows given in the matrix.

Example: Given an array **arr[1.....10][1.....15]** with a base value of **100** and the size of each element is **1 Byte** in memory find the address of **arr[8][6]** with the help of column-major order.

Solution:

Given:

Base address $B = 100$

Storage size of one element store in any array $W = 1$ Bytes

Row Subset of an element whose address to be found $n1 = 8$

Column Subset of an element whose address to be found $n2 = 6$

Lower Limit of row/start row index of matrix $LR = 1$

Lower Limit of column/start column index of matrix $= 1$

Number of column given in the matrix $N1 = \text{Upper Bound} - \text{Lower Bound} + 1$

$$= 10 - 1 + 1$$

$$= 10$$

Formula: used

Address of $A[n1][n2] = B + W * ((n2 - LC) * N1 + (n1 - LR))$

Address of $A[8][6] = 100 + 1 * ((6 - 1) * 10 + (8 - 1))$

$$= 100 + 1 * ((5) * 10 + (7))$$

$$= 100 + 1 * (57)$$

Address of $A[I][J] = 157$

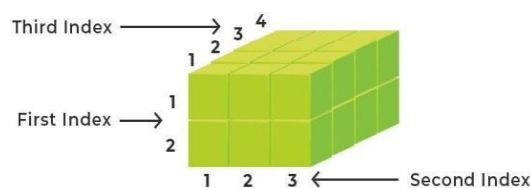
From the above examples, it can be observed that for the same position two different address locations are obtained that's because in row-major order movement is done across the rows and then down to the next row, and in column-major order, first move down to the first column and then next column. So both the answers are right.

So it's all based on the position of the element whose address is to be found for some cases the same answers is also obtained with row-major order and column-major order and for some cases, different answers are obtained.

3. Calculate the address of any element in the 3-D Array:

A **3-Dimensional** array is a collection of 2-Dimensional arrays. It is specified by using three subscripts:

1. Block size
2. Row size
3. Column size



Three-Dimensional Array
with 24 Elements

More dimensions in an array mean more data can be stored in that array.

3D Array

Given:

Base Address: 6500

3D Array : $A[50][60][20]$
 $N_1 \ N_2 \ N_3$

Element to be find: $A[34][50][15]$
 $n_1 \ n_2 \ n_3$

Row Major Order:

Base Address + $(n_1 N_2 N_3 + n_2 N_3 + n_3) * \text{Size}$

Column Major Order:

Base Address + $(n_3 N_2 N_1 + n_2 N_1 + n_1) * \text{Size}$

To find the address of any element in 3-Dimensional arrays there are the following two ways-

- Row Major Order
- Column Major Order

1. Row Major Order:

To find the address of the element using row-major order, use the following formula:

$$\text{Address of } A[n_1][n_2][n_3] = B + W * (N_2 * N_3(n_1 - x) + N_3(n_2 - y) + (n_3 - z))$$

Here:

B = Base Address (start address)

W = Weight (storage size of one element stored in the array)

N_1 = Width (total number of cells depth-wise)

N_2 = Row (total number of rows)

N_3 = Column (total number of columns)

x = Lower Bound of Plane

y = Lower Bound of Row

z = Lower Bound of Colom

Example: Given an array, **arr[1:9, -4:1, 5:10]** with a base value of **400** and the size of each element is **2 Bytes** in memory find the address of element **arr[5][-1][8]** with the help of row-major order?

Solution:

Given:

Row Subset of an element whose address to be found $n_1 = 5$

Column Subset of an element whose address to be found $n_2 =$

-1 Block Subset of an element whose address to be found $n_3 = 8$

Base address $B = 400$

Storage size of one element store in any array (in Byte) $W = 2$

Lower Limit of row/start row index of matrix $x = 1$

Lower Limit of column/start column index of matrix $y = -4$

Lower Limit of blocks in matrix $z = 5$

$$N2 = \text{Upper Bound} - \text{Lower Bound} + 1 = 1 - (-4) + 1 = 6$$

$$N3 = \text{Upper Bound} - \text{Lower Bound} + 1 = 10 - 5 + 1 = 6$$

Formula used:

$$\text{Address of } A[n1][n2][n3] = B + W * (N2 * N3(n1-x) + N3 * (n2-y) + (n3-z))$$

Solution:

$$\begin{aligned} \text{Address of } arr[5][-1][8] &= 400 + 2 * \{[6 * 6 * (5 - 1)] + 6 * [(-1 + 4)]\} + [8 - 5] \\ &= 400 + 2 * ((4 * 6 + 3) * 6 + 3) \\ &= 400 + 2 * (165) \\ &= 730 \end{aligned}$$

2. Column Major Order:

To find the address of the element using column-major order, use the following formula:

Column Major Order:

$$\text{Base Address} + (n3N2N1 + n2N1 + n1) * \text{Size}$$

$$\text{Address of } A[n1][n2][n3] = B + W(N2 * N1(n3 - x) + N1 * (n2 - z) + (n1 - y))$$

Here:

B = Base Address (start address)

W = Weight (storage size of one element stored in the array)

M = Row (total number of rows)

N = Column (total number of columns)

P = Width (total number of cells depth-wise)

x = Lower Bound of Plane

y = Lower Bound of Row

z = Lower Bound of Colom

Example: Given an array **arr[1:8, -5:5, -10:5]** with a base value of **400** and the size of each element is **4 Bytes** in memory find the address of element **arr[3][3][3]** with the help of column-major order?

Solution:

Given:

Row Subset of an element whose address to be found $I = 3$

Column Subset of an element whose address to be found $J = 3$

Block Subset of an element whose address to be found $K = 3$

Base address $B = 400$

Storage size of one element store in any array(in Byte) $W = 4$

Lower Limit of row/start row index of matrix $x = 1$

Lower Limit of column/start column index of matrix $y = -5$

Lower Limit of blocks in matrix $z = -10$

$$M = \text{Upper Bound} - \text{Lower Bound} + 1 = 5 + 5 + 1 = 11$$

$$N = \text{Upper Bound} - \text{Lower Bound} + 1 = 5 + 10 + 1 = 16$$

Formula used:

$$\text{Address of } [i][j][k] = B + W(M * N(i - x) + M * (k - z) + (j - y))$$

Solution:

$$\begin{aligned} \text{Address of } arr[3][3][3] &= 400 + 4 * \{[(3 - 1)] * 16 + [3 + 10] \} * 11 + [3 + 5] \\ &= 400 + 4 * ((32 + 13) * 11 + 8) \end{aligned}$$

$$\begin{aligned}
 &= 400 + 4 * (503) \\
 &= 400 + 2012 \\
 &= 2412
 \end{aligned}$$

2) 4D Array

Given:

Base Address: 7500

4D Array : $A[N_1][N_2][N_3][N_4]$

Element to be find: $A[n_1][n_2][n_3][n_4]$

Row Major Order:

Base Address + $(n_1N_2N_3N_4 + n_2N_3N_4 + n_3N_4 + n_4) * \text{Size}$

Column Major Order:

Base Address + $(n_4N_1N_2N_3 + n_3N_2N_1 + n_2N_1 + n_1) * \text{Size}$

1. Consider the linear arrays AAA [5 : 50], BBB [- 5 : 10] and CCC [1 : 8].
 - a) Find the number of elements in each array.
 - b) Suppose base (AAA) = 300 and $w = 4$ words per memory cell for AAA. Find the address of AAA [15], AAA [35] and AAA [55].
2. Suppose multidimensional arrays P and Q are declared as P(- 2: 2, 2: 22) and Q(1: 8, - 5: 5, - 10 : 5) stored in column major order
 - i. Find the length of each dimension of P and Q.
 - ii. The number of elements in P and Q.
 - iii. Assuming base address (Q) = 400, $W = 4$, find the effective indices E1 , E2 , E3 and address of the element Q[3, 3, 3].
3. Consider a multi-dimensional Array A[90] [30] [40] with base address starts at 1000. Calculate the address of A[10] [20] [30] in row major order and column major order. Assume the first element is stored at A[2][2][2] and each element take 2 byte.
4. Suppose multidimensional array P is declared as P(-20:20, 10:35) stored in Column major order Find the length of each dimension of P and the number of elements in P . Assume base address 500, $W=1$, find the effective address of P[0][30].

5. Consider 6 x 5 x 7 array, A which stores 64-bit hash values. The element A[2][3][4] is stored at an address location 2768 in row major order. Calculate the base address of A and compute the address of an element at A[3][3][3] using column-major ordering. Clearly mention the expressions for the above ordering used in the computations.

A[6][5][7]

N1 = 6 n1 = 2

N2 = 5 n2 = 3

N3 = 7 n3 = 4

w = 8 b = ?? (base address)

Row-Major Ordering A(2,3,4) = 2768

Addr(A(n1,n2,n3)) = ((n1 x N2 x N3) + (n2 x N3) + n3) x w + b

2768 = (2 * 5 * 7 + 3 * 7 + 4) * 8 + b

= 760 + b

base addr(A), b = 2008

6. Consider array stores marks of 30 students in 4 subjects. Row represents subjects and Columns represents students. Base Address of the array is 2500. Lower bound for row and column is 0.

a. Compute the address of marks [3][5] in row major order.

b. Compute the address of marks[2][20] in column major order.

7. Consider array stores marks of 70 students in 3 subjects. Row represents subjects and Columns represents students. Base Address of the array is 2500. Lower bound for row and column is 0 :

(a) Compute the address of marks[0][20] in row major order.

(b) Compute the address of marks[0][20] in column major order.